

CHAPTER 1

INTRODUCTION

1.1. Preamble

This paper presents an optimized 8×8 Booth multiplier architecture that is delay-efficient and energy-efficient for performance improvement in contemporary computing hardware like processors and AI accelerators. The design addresses some of the most significant challenges in generating partial products, reducing them, and summing them up with fewer circuits, logic reduction, and special compressors with certain characteristics. The major contributions include the use of inverters and a low-sign selection unit instead of the typical binary-to-two's complement circuits, elimination of redundant logic from Booth encoders, and use of low-complexity carry-save compressors. The summation is improved with an optimized Carry Look-Ahead Adder. The design was synthesized using a 45 nm CMOS technology and has better delay, power, and area usage than current designs, making it particularly well-suited for low-power and high-speed digital arithmetic computations.

1.2. Overview

The Booth multiplier is a widely used technique in digital arithmetic circuits, especially for signed binary multiplication. It enhances efficiency by reducing the number of partial products generated during multiplication. The radix-4 Booth algorithm, in particular, processes three bits of the multiplier at a time, allowing for faster multiplication by encoding values such as 0, $\pm b$, and $\pm 2b$. This approach significantly reduces the complexity and improves the speed of the multiplication operation compared to conventional methods. Booth multipliers are commonly used in digital signal processors (DSPs), arithmetic logic units (ALUs), and AI accelerators, where fast and energy-efficient arithmetic operations are critical.

The paper and project presented here suggest an optimized design of an 8×8 radix-4 Booth multiplier with a focus on delay, power, and area efficiency. The design employs various architectural optimizations like a reduced partial product generation stage using simple inverters and a compact sign selection unit in place of complex binary-to-two's complement converters. The optimizations are achieved in the compression stage using custom-designed signed compressors and in the summation stage using an optimized Carry Look-Ahead Adder.

1.3. Significance

The importance of this project goes beyond simple optimization, it represents a holistic architectural and microarchitectural innovation in Booth multiplier design, which is a building block in digital signal processing, embedded systems, and AI hardware accelerators. A major contribution is the presentation of a simplified but very efficient signed number handling approach with basic inverters and an extremely minimalist sign selection unit. This significantly reduces delay, power consumption, and the number of transistors needed, thereby solving one of the most prevalent bottlenecks in multiplier design.

Additionally, the use of smart Booth encoders eliminates duplicate logic, simplifying the process of generating partial products. During the compression stage, the project suggests innovative signed compressors such as signed half adders and application-specific 3:2 and 4:2 compressors that are optimized to deal with two's complement logic without overhead in delay or power. These compressors are designed to minimize the number of logic levels and ease the wiring, further optimizing the overall performance of the multiplier.

In the last phase, the optimized Carry Look-Ahead Adder (OCLA) blocks exhibit a careful design that restricts fan-in logic complexity without compromising on fast carry propagation, particularly in high bit-width designs. These enhancements, when compared collectively, exhibit considerable improvements in all performance parameters: the delay is decreased by more than 21%, power usage by almost 5%, and the area by over 5% compared to earlier state-of-the-art implementations.

1.4. Relevance

The relevance of this project is that it has direct applicability to current digital systems that require power-efficient arithmetic processing with low power and area requirements. Since Booth multipliers are the fundamental blocks in central processing units (CPUs), digital signal processors (DSPs), and AI accelerators, it is essential to optimize their performance in order to enhance the efficiency of these systems. With the exponential increase in data-intensive applications like machine learning, edge computing, real-time image and video processing, and embedded systems, there is an urgent requirement for fast and energy-efficient multipliers.

This project fills that requirement by introducing an 8×8 Booth multiplier architecture that improves major performance parameters like delay, power, and area considerably without adding design complexity. Its timeliness is further increased by its realization in 45 nm CMOS-technology, thus it clearly addresses today and tomorrow's integrated circuit designs. Moreover, as energy efficiency emerges as a pervasive design constraint in portable devices as well as giant data centers, the minimized power-delay product (PDP) and area-delay product (ADP) through this design render it a very appealing solution.

1.5. Organization of Report

The major project report is organized into 5 chapter

Chapter 1 provides a brief introduction of our domain, the significance and relevance of our project. It provides the reader with a comprehensive understanding of the report's context, the issues it addresses, and the goals it aims to achieve.

Chapter 2 provides the literature review of the existing research and scholarly works relevant to the project, providing a foundation for understanding the current state of knowledge in the field. It deals with the existing technologies of the field providing an overview of the domain and research gaps.

Chapter 3 provides the methodology employed in the study, outlining the systematic approach taken to achieve the research objectives.

Chapter 4 provides the findings of the study, detailing the implementation process and analyzing the results obtained.

Chapter 5 provides a comprehensive summary of the study's findings and their implications, concluding the research with a reflection on the outcomes and directions for future work.

CHAPTER 2

LITERATURE SURVEY

2.1. Introduction

The literature survey chapter provides a comprehensive overview of existing research, methodologies, and technological advancements relevant to the topic under investigation. This chapter aims to establish the context for the current study by critically analyzing previous works, identifying key trends, and highlighting gaps or challenges that persist in the field. By systematically reviewing and synthesizing the available literature, this section lays the foundation for understanding the motivations behind the present research and justifies the need for further exploration. This chapter will explore the evolution of multiplier architectures, examine different approaches to partial product generation, reduction, and summation, and assess the effectiveness of proposed optimizations in addressing key performance metrics.

2.2. Survey based on different technologies used in research.

Y. Mounica et al. (2021) – "Energy Efficient Signed and Unsigned Radix-16 Booth Multiplier Design" In this paper, Y. Mounica and her co-authors present a radix-16 Booth multiplier that supports both signed and unsigned operations while aiming to improve energy efficiency. The proposed multiplier achieves significant energy savings—up to 31.30% in pipelined mode and 27.79% in non-pipelined mode.

Tingting Zhang et al. (2022) – "Design of Majority Logic-Based Approximate Booth Multipliers for Error-Tolerant Applications" Tingting Zhang and colleagues investigate approximate radix-4 Booth multipliers using majority logic (ML), a design paradigm particularly suited for emerging nanotechnologies such as nanomagnetic logic and MTJ devices. These techniques result in substantial improvements in power and area efficiency.

Mingtao Zhang et al. (2023) – "Evaluation of Application-Independent Unbiased Approximate Multipliers on Quantized CNNs" Mingtao Zhang and his team focus on creating unbiased approximate 8-bit multipliers tailored for use in quantized convolutional neural networks (CNNs). They introduce a novel "double-carry 4-2" approximate compressor to ensure zero

mean error (Zero-ME), aiming to reduce hardware cost while maintaining application accuracy.

N.V.V.K. Boppana and S. Ren (2023) – "Simple Low Power-Delay-Product Parallel Signed Multiplier Design Using Radix-8 Structure" Naga Venkata Vijaya Krishna Boppana and Saiyu Ren introduce a radix-8 based parallel signed multiplier. The architecture simplifies partial product reduction using optimized grouping and BEC-like (Binary Excess-1 Code) structures, leading to faster and lower power designs, particularly effective for larger bit-widths.

Ziang et al. (2016)- The authors implemented an approximate radix-8 Booth multiplier for DSP applications, accepting minor inaccuracies for power and area savings. This approach inspired encoding simplification. Your work adopts radix-4 for better complexity control and ensures accuracy while maintaining high energy efficiency.

Silveira et al. (2017)- This paper introduced power-efficient adder compressors (such as optimized 4:2 compressors) for motion estimation in video processing. Their low-delay compressor design directly influenced your development of Signed Half Adders (SHA) and custom 3:2 compressors for the Booth compression stage.

Xue et al. (2018)- The study designed a radix-4 Booth multiplier using an optimized binary-to-two's complement (B-to-C) converter, resulting in a reduced power-delay product. Your design replaces the B-to-C block with inverters and sign selection logic, achieving even better delay and area performance with fewer transistors.

Boppana et al. (2019)- This paper proposed a low-cost, high-performance 8×8 Booth multiplier using efficient partial product accumulation and parallelism in 32nm CMOS. It served as a benchmark in your study. Your 45nm-based design outperforms it in delay, power, and area metrics while maintaining scalability.

Shah et al. (2021)- The authors explored Single Electron Transistor (SET)-based multipliers operating at sub-threshold voltages, achieving ultra-low power consumption. However, SET fabrication is still emerging. Your design remains grounded in CMOS 45nm technology, ensuring practical synthesis for commercial ASIC flows.

Zhang et al. (2023)- This work evaluated approximate multipliers in quantized CNNs and showed that unbiased arithmetic errors can preserve neural network accuracy while reducing energy. It supports the use of your exact, low-delay Booth multiplier in AI accelerators where deterministic performance is critical.

Hasan & Kort (2007)- The authors applied formal verification techniques to Wallace tree multiplier synthesis, ensuring correctness in partial product reduction. This reinforces the importance of structured and validated compression logic—an approach also emphasized in your carry-save compression architecture.

Edavoor et al. (2024)- This paper presented a hybrid radix-4/radix-8 Booth encoder for floating-point multiplication in image processing systems. It balanced encoding flexibility and performance for FP applications. While your work targets integer arithmetic, it aligns with the trend and is scalable for hybrid encoders in future designs.

Table 2.1: Different technologies and approaches used in research

Year, Title and Author	Methodology	Key Findings	Relevance to current work
2023 Simple low power-delay-product parallel signed multiplier design using radix-8 structure. Boppana et al.	Designed a novel radix-8 multiplier with precomputed trivial/non-trivial values, and efficient partial product reduction using optimized grouping and BEC-like structures.	Lowered area, power, and delay. Achieved improved Power Delay Product (PDP), particularly in higher-bit multipliers.	Highlights radix-8 optimizations using BEC logic. Our design uses radix-4 to reduce complexity
2023 Evaluation of Application-Independent Unbiased Approximate Multipliers on Quantized CNNs Zhang et al.	Introduced Zero-ME 8-bit multipliers using double-carry 4-2 compressors. Evaluated them on quantized CNNs to test application-independent metrics like ME (Mean Error).	Demonstrated potential of Zero-ME multipliers for CNNs with unbiased error profiles and competitive power savings.	Focuses on power-efficient approximate multipliers. It similarly uses optimized compressor logic to enhance efficiency, and is adaptable to CNN accelerators.
2022 Design of Majority Logic-Based Approximate Booth Multipliers for Error-Tolerant Applications Zhang et al.	Developed approximate radix-4 Booth multipliers using Majority Logic (ML). Proposed various PP generation methods producing both single-sided and double-sided errors.	Enabled error-tolerant, low-power designs for nanotechnologies. Significant reductions in area and power shown for ML-based implementations.	Uses majority logic for low-power approximate Booth multipliers, work follows a similar logic simplification path but ensures exact computation with minimal transistor overhead.

2021 Energy efficient signed and unsigned radix 16 booth multiplier design Y. Mounica et al.	Introduces an energy-efficient radix-16 Booth multiplier supporting both signed and unsigned operations.	Achieved energy improvements up to 31.30% (pipelined) and 27.79% (non-pipelined). Radix-16 design reduced partial product count significantly.	Achieves power savings by reducing partial products via radix-16. Our design opts for radix-4 to avoid encoder complexity while retaining area and delay efficiency.
2016 Approximate radix-8 Booth multipliers for low-power and high-performance operation Ziang et al.	Implemented radix-8 Booth encoding with approximate arithmetic units. Targeted low-power DSPs.	Reduced the number of partial products, saving area and power. Accepted minor arithmetic inaccuracies.	Influenced encoding simplicity in your radix-4 Booth multiplier. work chooses radix-4 for a better complexity-performance balance.
2017 Adder compressors for motion estimation Silveira et al.	Custom-designed compressors (e.g., optimized 4:2) for sum-of-absolute-differences (SAD) operations.	Developed low-delay, power-efficient adders tailored to video processing hardware.	Their compressor structure influenced the design of your optimized S3:2C and SHA units for compression.
2018 Low-power-delay-product radix-4 Booth multiplier Xue et al.	Designed a radix-4 Booth multiplier using enhanced B-to-C converter logic. Synthesized for CMOS.	Reported low power-delay product (PDP) and transistor count. Demonstrated benefits of optimized encoding logic.	Closely related benchmark. Our inverter-based PPG offers even better delay and PDP with reduced transistor count.
2019 Low-cost high-performance Booth multiplier Boppana et al.	Used parallelism and efficient partial product accumulation to improve throughput.	Demonstrated competitive performance at low hardware cost in 32nm CMOS.	Used as a benchmark in our performance tables; your design surpasses it in power, area, and delay at 45nm.
2021 SET-based 8×8 multipliers Shah et al.	Leveraged Single Electron Transistor (SET) technology for sub-threshold operation.	Achieved ultra-low power but required emerging fabrication techniques.	Highlights futuristic directions; your design remains grounded in scalable 45nm CMOS for practical synthesis.
2023 Unbiased approximate multipliers on CNNs Zhang et al.	Tested approximate multipliers in quantized convolutional neural networks.	Found that unbiased errors preserved learning accuracy while saving energy.	Supports application of your low-delay Booth multiplier in AI accelerators, particularly with quantized models.
2007 Formal synthesis of Wallace tree multipliers Hasan & Kort	Applied formal methods to automate synthesis of Wallace tree structures. Verified correctness of reduction trees.	Ensured reliable synthesis of compression logic in digital multipliers.	Validated the importance of structured and verifiable compression schemes similar to your carry-save approach.

2024 Hybrid Radix-4/8 Booth encoder for FP applications Edavoor et al.	Combined radix-4 and radix-8 encoding for floating-point multiplication in image systems.	Balanced encoding flexibility and compression stages for improved FP operations.	Although you focus on integer 8×8 designs, this work shows future potential of hybrid encoders — your radix-4 approach is scalable toward such enhancements.
--	---	--	--

2.3. Survey based on delay and power results.

Table 2.2: Delay and power result obtained

Year, Title and Author	Power and Delay obtained	Limitations	Measures taken
2018 Low-Power-Delay-Product Radix-4 8×8 Booth Multiplier in CMOS Boppana et al.	Achieved 49.46% power reduction and 7.08% delay reduction using 90nm CMOS	Optimizations limited to early pipeline stage	Introduced binary-to-complement converter and 2:1 MUX in first stage
2023 Simple Low Power-Delay-Product Parallel Signed Multiplier Boppana et al.	Reduced PDP using radix-8 encoding and XOR-XNOR logic	Pre-computation stage and logic control complexity	Used hybrid adders and simplified partial product encoding
2021 SET-Based 8×8 Signed Multipliers Shah et al.	SET Booth: 1.41 mW, 6.09 ps; CMOS Booth: 7.53 mW, 454.94 ps	Fabrication complexity, CMOS-SET interfacing	Implemented and simulated in Cadence Virtuoso for comparison
2021 Approximate Wallace Tree Multiplier Bhardwaj et al.	Power reduced by 41.96%, area by 34.49%, latency by ~24%	Approximation-induced minor accuracy loss	Applied carry-in prediction and Wallace Tree optimization
2013 Radix-4/8 Multi-Modulus Booth Multipliers Muralidharan et al.	Area reduced by ~60%, delay increased by 13–18%, 5% higher power	Delay overhead due to MUX sharing	Reused hardware for multiple moduli in RNS environment
2007 Automated Formal Synthesis of Wallace Tree Multipliers Hasan et al.	Focused on correctness verification rather than PDP metrics	No direct power/delay improvements	Used theorem-proving to ensure logic-correct netlist generation
2023 Evaluation of Application-Independent Unbiased Approximate Multipliers Zhang et al.	Power = 9.69 μ W, delay = 4.91 ns, PDP = 47.57 fJ (Zero-ME 8-7)	Zero-mean-error alone insufficient to gauge CNN performance	Designed double-carry 4-2 compressor and tested on CNNs
2022 Majority Logic-Based Approximate Booth Multipliers Zhang et al.	Up to 53% ADP improvement and 20% reduction in delay	Biased single-sided errors in some configurations	Introduced unbiased double-sided PP generators using ML

2018 Booth Multiplier with Optimised Power Delay Product Chaitanya et al.	25.9% delay reduction and ~7% PDP improvement (45nm)	Sign extension complexity and partial product summation	Used inverter-based sign extension and custom 10T full adder
2015 Approximate Radix-8 Booth Multipliers Jiang et al.	Delay reduced by 20%, power by up to 69% with truncation	Accuracy degraded with aggressive truncation	Designed approximate recoding adder and applied Wallace tree
2018 Hardware-Efficient Approximate Booth Multiplier Xue et al.	Reduced PDP and area via logic simplification in Booth encoding	Limited scalability to larger bit-widths	XOR-based Booth encoder and MUX-controlled logic used
2011 High-Accuracy Fixed-Width Modified Booth Multipliers for Lossy Applications Wang et al.	12.3% (16-bit) and 6.3% (8-bit) reductions in mean-square error; minor delay/area increase	Traditional compensation techniques ineffective for Booth variants	Introduced simplified sorting-network-based compensation circuit, achieving better PSNR in DWT and RGB-YUV conversion

Xue et al. (2018) – Low-Power-Delay-Product Radix-4 8×8 Booth Multiplier in CMOS: This paper proposes an optimized radix-4 Booth multiplier that reduces both power consumption and delay by introducing a binary-to-two's complement converter and a 2:1 multiplexer. The design, implemented in 90nm CMOS, achieves 49.46% power and 7.08% delay reduction. It focuses on optimizing the early pipeline stages.

Boppana et al. (2023) – Simple Low Power-Delay-Product Parallel Signed Multiplier: The authors present a radix-8 multiplier using XOR-XNOR logic for partial product generation and hybrid adders for final summation. This architecture significantly reduces the power-delay product. Though pre-computation is complex, the approach balances speed and energy efficiency.

Shah et al. (2021) – SET-Based 8×8 Signed Multipliers: This paper compares Booth, Array, and Baugh-Wooley multipliers using Single Electron Transistor (SET) and CMOS technologies. The SET-based Booth multiplier demonstrates significantly lower power and delay than its CMOS counterpart. Challenges include fabrication complexity and CMOS-SET integration.

Bhardwaj et al. (2021) – Approximate Wallace Tree Multiplier: The proposed design reduces power by 41.96%, area by 34.49%, and latency by 24% using approximation in Wallace Tree structures. It is well-suited for error-resilient applications, with minor trade-offs in accuracy. Carry-in prediction and bit-width awareness enhance efficiency.

Muralidharan et al. (2013) – Radix-4 and Radix-8 Multi-Modulus Booth Multipliers: This work introduces multi-modulus multipliers using radix-4 and radix-8 Booth encoding in Residue Number Systems. The design reuses hardware for multiple operations, saving nearly 60% area but incurring a 13–18% delay increase and minor power penalty.

Hasan et al. (2007) – Automated Formal Synthesis of Wallace Tree Multipliers: This paper emphasizes correctness over performance by using theorem-proving to formally verify the synthesis of Wallace Tree multipliers. Although it doesn't optimize for power or speed, it guarantees logical accuracy across abstraction levels, making it suitable for critical systems.

Zhang et al. (2023) – Evaluation of Application-Independent Unbiased Approximate Multipliers on CNNs: The authors propose Zero-Mean Error (Zero-ME) 8-bit approximate multipliers using a double-carry 4-2 compressor. Their approach achieves 9.69 μ W power and 4.91 ns delay. While unbiasedness improves CNN results, ME alone doesn't guarantee optimal performance, highlighting the need for application-specific metrics.

Zhang et al. (2022) – Majority Logic-Based Approximate Booth Multipliers: This design uses majority logic to construct radix-4 Booth multipliers optimized for nanotechnology-based circuits. Various partial product generators with single- and double-sided errors are explored. The designs are efficient for image processing and neural network tasks requiring flexible accuracy.

Chaitanya et al. (2018) – Booth Multiplier with Optimised Power Delay Product: A custom 10-transistor full adder and carry-skip adder are used in this design to enhance speed and reduce PDP under 45nm technology. The architecture improves delay by 25.9%, but complexity increases due to sign extension and partial product management.

Jiang et al. (2015) – Approximate Radix-8 Booth Multipliers: This paper introduces two approximate radix-8 Booth multipliers using an approximate recoding adder and fixed-width truncation. The designs reduce delay by 20% and power by up to 69%, making them suitable for low-power DSP applications at the cost of minor accuracy loss.

Xue et al. (2018) – Hardware-Efficient Approximate Booth Multiplier: A hardware-lean Booth multiplier is proposed using simplified Booth encoding and multiplexer logic. The design

offers low power-delay product and area, but scalability is limited for higher operand widths. It emphasizes early-stage logic simplification for efficiency.

Wang et al. (2011) – High-Accuracy Fixed-Width Modified Booth Multipliers for Lossy Applications: This work presents a fixed-width modified Booth multiplier with a sorting-network-based error compensation function. It reduces mean-square error by over 12% and improves PSNR in real-world applications like DWT and RGB-YUV conversion. Hardware overhead remains minimal.

2.4. Problem Statement

- Multipliers are critical in digital systems, but radix-4 Booth multipliers still face efficiency challenges.
- High delay, power consumption, and area usage are major limitations in existing designs.
- Binary-to-two's complement circuits add unnecessary complexity and delay in partial product generation.
- Booth encoding and multiplexer logic often include redundant components, reducing performance.
- Conventional compressors in the reduction stage increase delay and power overhead.
- Final adders with large fan-in logic lead to slower performance and greater area usage.

2.5. Summary

Multipliers are critical components in digital systems, particularly in processors and AI hardware, where high-speed and efficient arithmetic operations are essential.. However, conventional Booth multipliers often face challenges related to delay, power consumption, and area efficiency mainly during partial product generation, compression, and final summation. To overcome these limitations, this project introduces a fully optimized radix-4 Booth multiplier that simplifies signed logic, incorporates efficient compressors, and utilizes a high-speed final adder. The proposed design achieves better performance in terms of speed, power, and area, making it ideal for modern high-performance and low-power applications.

CHAPTER 3

METHODOLOGY

3.1. Introduction

This chapter presents the detailed methodology used to design and implement an optimized radix-4 Booth multiplier that focuses on reducing delay, power consumption, and area utilization. The proposed approach targets all three major stages of the multiplier architecture: partial product generation, partial product reduction (compression), and final summation. Each stage is systematically enhanced through architectural and circuit-level optimizations. Simplified logic circuits, such as inverters and sign selection units, are used in place of conventional binary-to-two's complement converters to handle signed numbers efficiently. Custom-designed compressors are introduced to accelerate partial product reduction with minimal delay and power overhead. Finally, an optimized Carry Look-Ahead Adder (OCLA) is employed in the final addition stage to achieve fast computation with reduced complexity.

3.2. Objectives

- To design a radix-4 Booth multiplier architecture that supports signed 8×8 multiplication
- To minimize critical path delay by replacing conventional binary-to-two's complement circuits with a simplified design using inverters and a sign selection unit.
- To reduce power consumption and area utilization by introducing optimized Booth encoders
- To validate the proposed Booth multiplier design through simulation and synthesis, and to compare its performance with existing designs based on delay, power, area, power-delay product (PDP), and area-delay product (ADP).

3.3. VLSI Design Flow

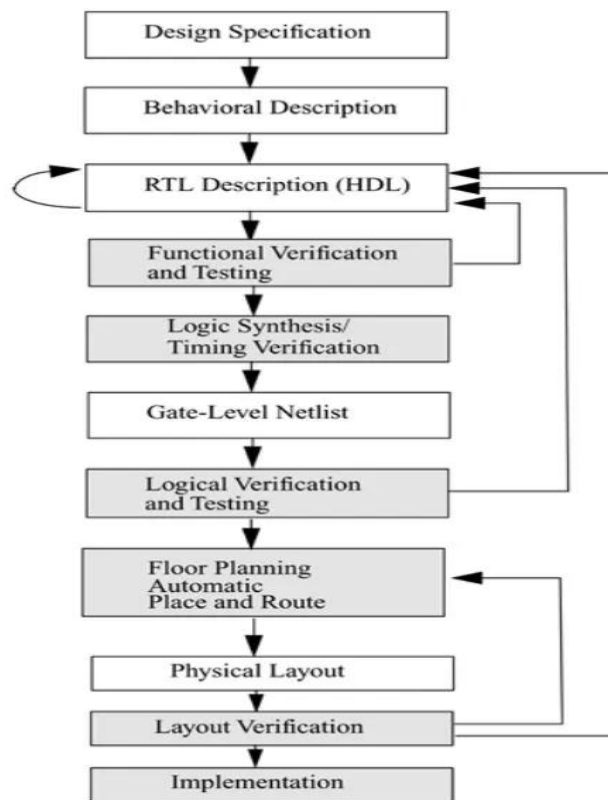


Fig 3.1: VLSI Design Flow

The VLSI design flow is a structured process that transforms a high-level idea into a physical integrated circuit. It begins with the design specification, which defines the circuit's functionality and architecture. This is followed by a behavioral description, usually written in a Hardware Description Language (HDL), to model the system's behavior. The behavioral model is then converted into an RTL (Register Transfer Level) description, also in HDL, specifying data flow and control logic. This RTL model undergoes functional verification and testing to ensure correctness. Next, logic synthesis and timing verification are performed to convert the RTL into a gate-level netlist that meets design constraints. The netlist is then subjected to logical verification and testing. After verification, floor planning and automatic place and route processes are used to generate the physical layout of the chip. This layout undergoes layout verification to detect any design rule violations or mismatches. Finally, the verified layout proceeds to implementation, where the design is fabricated. Throughout the flow, feedback loops allow designers to return to previous stages for refinement, making the process iterative and enabling optimization at various levels.

3.3.1 Design Specification

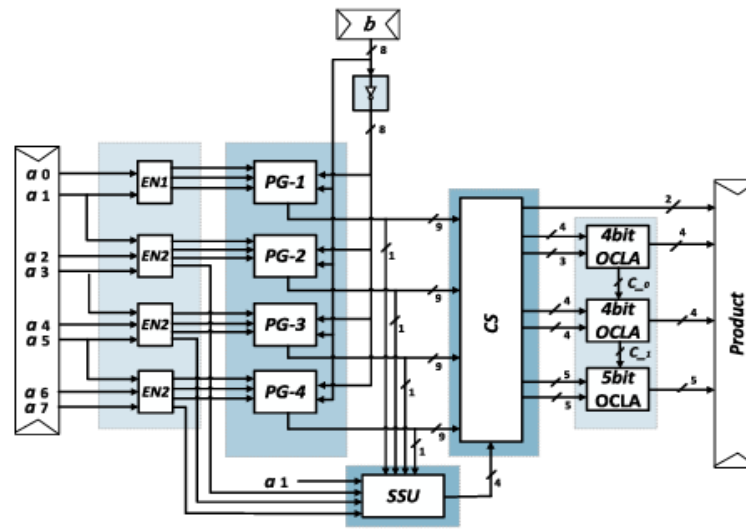


Fig 3.2: Proposed design architecture of Radix-4 8×8 Booth multiplier

The goal of this project is to design an optimized 8×8 radix-4 Booth multiplier architecture that minimizes critical path delay, power consumption, and silicon area, making it suitable for high-performance, low-power ASIC applications. The multiplier is built to handle signed inputs and targets applications such as digital signal processors (DSPs), embedded systems, and AI accelerators. To achieve this, the design introduces innovations across all stages of the multiplier: partial product generation, compression, and final summation. The partial product generation is optimized using simple inverters and a compact sign selection unit, replacing traditional binary-to-two's complement logic to reduce delay and hardware complexity.

Additionally, the Booth encoding stage uses custom encoders (EN1 and EN2) to eliminate redundant logic, improving area and speed. The compression stage incorporates signed compressors with minimal gate delay using only two XOR gates, reducing power and critical path delays. The final stage uses a 13-bit segmented Carry Look-Ahead Adder (OCLA) that is specifically structured to use smaller fan-in logic gates, further minimizing propagation delay and power consumption. The entire architecture is implemented in SystemVerilog and synthesized using Altera Quartus II. This design is functionally verified using testbenches in Modelsim, with synthesis results confirming its efficiency over prior designs in key metrics such as delay, power, and area.

3.3.2 Behavioral Description

Behavioral Description The behavioral modeling of the optimized 8×8 radix-4 Booth multiplier was carried out using SystemVerilog in Altera Quartus, with functional verification performed through simulation in ModelSim. At this stage, the focus was on describing the architecture at a high level, capturing the logical behavior of the multiplier without considering gate-level implementation details.

The design was structured into three main functional blocks: partial product generation, compression, and final summation. The Booth encoding logic (EN1 and EN2) was modeled to produce signed partial products based on overlapping triplets of multiplier bits. The sign selection unit and inverter-based two's complement logic were behaviorally implemented to reduce delay and logic complexity. Signed compressors (SHA, S3:2C, S4:2C, and O3:2C) were written to handle carry-save compression of partial products. Finally, the 13-bit Carry Look-Ahead Adder (OCLA) was coded in a modular fashion to complete the summation stage. The simulation results in ModelSim verified correct signed multiplication, with waveform outputs matching expected results for various test vectors. This confirmed the functional correctness of the multiplier's high-level behavior before synthesis

3.3.3 RTL Description (HDL)

The Register Transfer Level (RTL) design for the optimized Booth multiplier was developed using SystemVerilog, structured into modular components representing each functional stage. The design includes well-defined modules for the Booth encoders, partial product generator, sign selection unit, compressors, and the 13-bit final adder. Each module is parameterized for scalability and ease of verification. RTL coding followed best practices in hardware description, such as synchronous design principles, appropriate use of always blocks, and non-blocking assignments.

Special attention was given to the partial product generation stage, where inverters and sign selection logic replaced traditional binary-to-two's complement circuits. This simplification helped reduce the number of gates and the overall critical path delay. The compressors were also modeled using minimal XOR-based logic, and the final summation was implemented using a three-stage segmented Carry Look-Ahead Adder to enable faster additions. The

modular approach ensured design clarity, reusability, and better testbench integration in subsequent verification stages.

3.3.4 Functional Verification and Testing

After RTL modeling, functional verification was conducted using ModelSim. Custom testbenches were written in SystemVerilog to simulate the behavior of the multiplier. These testbenches included exhaustive test vectors for signed 8-bit inputs to validate correct Booth encoding, partial product generation, compression logic, and the final addition. The waveform outputs confirmed that all intermediate and final values matched the expected results for various corner cases, including positive, negative, and mixed sign inputs.

The simulation environment allowed real-time monitoring of signal transitions across all major modules. The simulation output demonstrated correct handling of signed multiplication operations, and the correct propagation of control signals like sign selection bits and carry signals. This stage was critical for identifying and correcting logical bugs early in the design flow, ensuring that the multiplier functioned correctly at the behavioral and RTL levels before synthesis.

3.3.5 Logic Synthesis / Timing Verification

The synthesized design was implemented using Cadence Genus with FreePDK 45nm CMOS process technology. RTL-to-gate synthesis involved translating the SystemVerilog code into a gate-level netlist, applying technology-specific logic cells. During this stage, synthesis constraints such as target frequency and input/output delays were defined to guide the tool in optimizing the logic for speed and area.

The synthesis reports provided key performance metrics, including total gate count, critical path delay, power consumption, and area utilization. Optimization strategies from the RTL, such as using inverters instead of full binary-to-two's complement logic, significantly improved synthesis results. Compared to previous designs, the synthesized Booth multiplier showed marked improvements in delay (21.46%), power (4.46%), and area (5.2%), validating the effectiveness of the design choices made during RTL modeling.

3.3.6 Gate-Level Netlist

The output of the synthesis process was a gate-level netlist, a structural description of the circuit using standard cells from the 45nm library. This netlist represents the actual logic that would be implemented on silicon. The netlist includes all logic gates, flip-flops, and interconnections, reflecting the optimized structure derived from the RTL design. The netlist is crucial for both formal equivalences checking and the backend design flow (place & route, layout).

Alongside the netlist, Standard Delay Format (SDF) files and power reports were generated. These are used in gate-level simulations to verify that the post-synthesis design still meets the original functional and timing requirements. At this stage, the design is ready to be passed into the physical design flow, although in this project, the focus remained on front-end optimization. The generation and validation of the gate-level netlist mark a key milestone in ensuring that the optimized Booth multiplier is both functionally correct and implementation-ready.

3.4. Software Requirements

3.4.1. Altera Quartus II

Altera Quartus II, now known as Intel Quartus Prime, is an advanced software suite developed by Intel (formerly Altera) for the design and implementation of digital logic circuits on FPGAs (Field-Programmable Gate Arrays) and CPLDs (Complex Programmable Logic Devices). It provides an all-in-one environment that supports hardware description languages like VHDL and Verilog, allowing users to describe complex digital systems textually. In addition to text-based design, the software offers graphical tools such as a block diagram editor and a state machine editor for users who prefer visual development methods.

One of the core features of Quartus Prime is its synthesis and optimization engine, which converts HDL code or schematic designs into gate-level representations while optimizing for speed, area, and power efficiency. The software includes a powerful fitting tool that maps the design onto the physical resources of the selected FPGA, handling placement, routing, and pin assignment automatically. For validating timing requirements, the integrated Time Quest

Timing Analyzer performs static timing analysis to ensure that the circuit operates correctly under specified conditions.

3.4.2 ModelSim Software

ModelSim is a leading HDL (Hardware Description Language) simulation tool developed by Mentor Graphics, now a part of Siemens EDA. It is widely used in the semiconductor and electronic design industry for simulating and verifying digital systems described in VHDL, Verilog, SystemVerilog, and mixed-language designs. ModelSim is known for its robust simulation engine, user-friendly GUI, and strong debugging capabilities, making it a preferred choice for both academic and industrial VLSI projects.

ModelSim allows designers to compile, simulate, and debug HDL code in a unified environment. It provides a powerful waveform viewer, variable monitoring, and breakpoint setting, which help identify design bugs early in the development process. With support for behavioral, RTL, and gate-level simulation, ModelSim is capable of verifying the logical correctness of digital circuits before synthesis and physical implementation.

3.5. Summary

The methodology for this project revolves around designing an optimized 8×8 radix-4 Booth multiplier targeting improvements in delay, power, and area. The main objective is to develop a high-speed and energy-efficient signed multiplier by optimizing all three key stages: partial product generation, compression, and final summation. The design introduces simplified hardware using inverter-based two's complement logic, customized Booth encoders, and low-complexity signed compressors, followed by a segmented Carry Look-Ahead Adder for fast final summation.

To achieve these goals, the VLSI design flow was followed up to the front-end stages. The process began with the RTL design in SystemVerilog, where each functional block was described and modularized for better clarity and reusability. Functional verification was performed using ModelSim, where waveforms confirmed the correctness of signed multiplication.

CHAPTER 4

RESULT ANALYSIS

4.1. Introduction

This chapter presents the analysis of results obtained from the implementation and simulation of the proposed radix-4 Booth multiplier design. The focus is on evaluating the performance of the multiplier in terms of key parameters such as delay, power consumption, and area utilization. These results are compared with existing Booth multiplier designs to highlight the improvements achieved through the proposed architecture. The design is synthesized and tested using Altera Quartus II Prime software, ensuring accurate analysis under real hardware constraints. By examining the performance metrics and comparing them with previous works, this chapter validates the effectiveness of the design choices made in each stage—partial product generation, compression, and final summation—and demonstrates the efficiency of the overall multiplier implementation.

4.2. Performance criteria

1. Delay

The timing analysis for the existing Booth multiplier was performed using the Quartus II TimeQuest Timing Analyzer. The key results are as follows:

- **Critical Data Path Delay:** The data path delay for the critical path in the existing Booth multiplier design is 3.242 ns.
- **Number of Logic Levels:** The critical path traverses 12 logic levels, indicating a relatively deep combinational logic path.
- **Setup Slack:** The setup slack is -2.325 ns. The negative slack indicates that the data path delay exceeds the allowed clock period, and the design would not function reliably at the intended clock frequency

5	-2.172	stage1_sum[0]	stage2_sum[13]	clk	clk	1.000	-0.088	3.089
---	--------	---------------	----------------	-----	-----	-------	--------	-------

Path #1: Setup slack is -2.325 (VIOLATED)							
Path Summary		Statistics	Data Path	Waveform	Extra Fitter Information		
	Property	Value	Count	Total Delay	% of Total	Min	Max
1	Setup Relationship	1.000					
2	Clock Skew	-0.088					
3	Data Delay	3.242					
4	Number of Logic Levels		12				
5	Physical Delays						
1	Arrival Path						
1	Clock						
1	IC		3	1.240	44	0.000	1.020
2	Cell		3	1.552	55	0.000	0.825
2	Data						
1	IC		13	0.808	24	0.000	0.808
2	Cell		14	2.124	65	0.000	0.666
3	uTco		1	0.310	9	0.310	0.310
2	Required Path						
1	Clock						
1	IC		3	1.189	44	0.000	0.978
2	Cell		3	1.500	55	0.000	0.825

Note: Negative delays are omitted from totals when calculating percentages

Fig. 4.1: Delay report of existing design

Path #1: Setup slack is -1.738 (VIOLATED)							
Path Summary		Statistics	Data Path	Waveform	Extra Fitter Information		
	Property	Value	Count	Total Delay	% of Total	Min	Max
1	Setup Relationship	1.000					
2	Clock Skew	-0.073					
3	Data Delay	2.663					
4	Number of Logic Levels		12				
5	Physical Delays						
1	Arrival Path						
1	Clock						
1	IC		4	2.882	67	0.000	1.219
2	Cell		4	1.402	32	0.000	0.649
2	Data						
1	IC		13	0.639	23	0.000	0.639
2	Cell		14	1.792	67	0.000	0.536
3	uTco		1	0.232	8	0.232	0.232
2	Required Path						
1	Clock						
1	IC		4	2.758	67	0.000	1.170
2	Cell		4	1.353	32	0.000	0.649

Note: Negative delays are omitted from totals when calculating percentages

Fig. 4.2: Delay report of proposed design

2. Power

The power consumption of conventional Booth multipliers was analyzed using the PowerPlay Power Analyzer tool in Quartus II. The following summarizes the results:

- The proposed hybrid Booth multiplier consumes less total power (66.14 mW) compared to the conventional Booth multiplier (71.50 mW).
- Both designs have similar core static power, but the proposed design has lower I/O power, contributing to its overall lower consumption.
- The reduction in logic elements (143 vs. 203) in the proposed design also supports lower power usage.

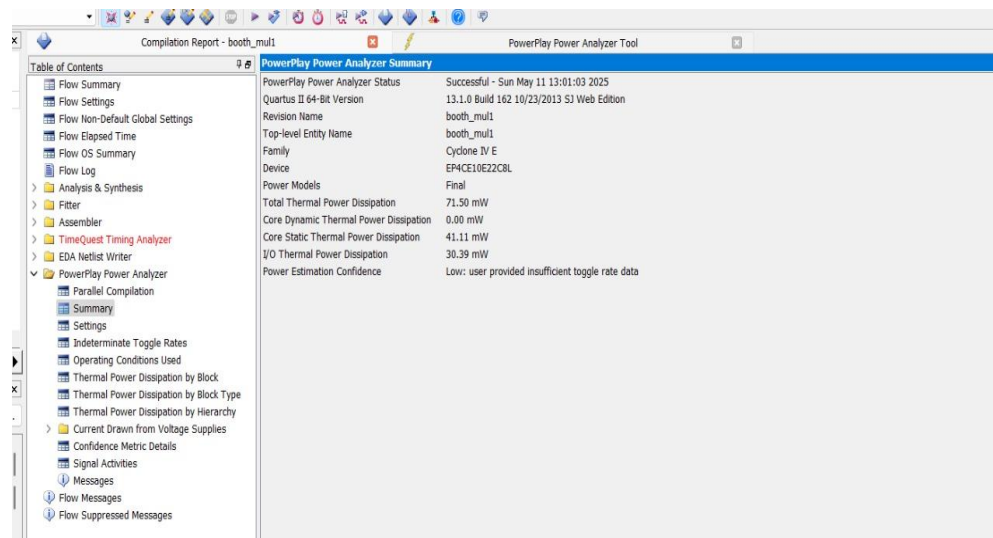


Fig. 4.3: Power consumption of existing work

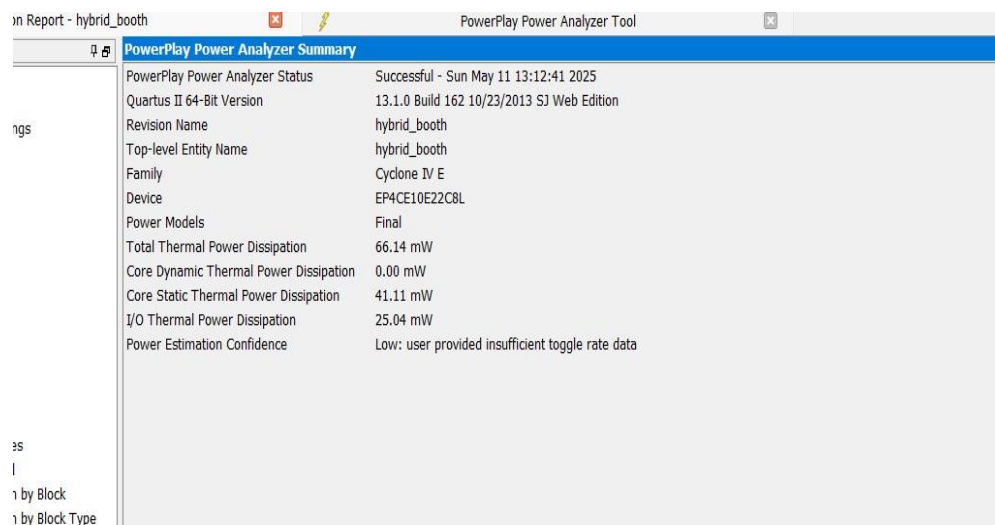


Fig. 4.4: Power consumption of proposed work

3. Area

The area utilization of the proposed Booth multiplier designs was analyzed using Quartus II software targeting the Cyclone IV E FPGA (EP4CE10E22C8L). The proposed design utilized 143 logic elements (1% of total) and 30 registers, while the conventional design required 203 logic elements (2% of total) with no dedicated registers. Both designs demonstrated efficient resource usage, with pin utilization at 37% and 35%, respectively. The results highlight the area efficiency of the proposed architectures.

4.3. Netlist and simulation results

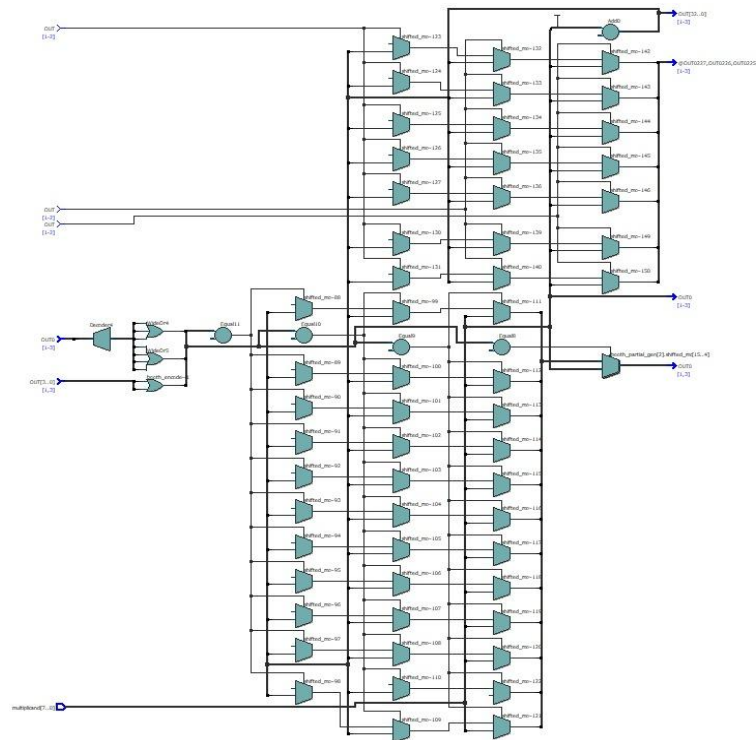


Fig. 4.5: RTL schematic of conventional booth multiplier

The netlist schematic shown represents the gate-level structure of a conventional Booth multiplier, which serves as a baseline implementation without delay or energy optimizations. The design consists of typical modules for Booth encoding, partial product generation, and summation. The Booth encoder extracts control signals from the multiplier input, guiding the generation of partial products via a series of multiplexers and logic gates. Each partial product is generated by modules such as `mux4_mno_iXX`, which handle sign extension, inversion, and appropriate bit shifting. These partial products are then passed through an unoptimized compression stage and combined using basic adder structures. Unlike optimized designs that use compressor trees and hybrid radix schemes to reduce delay and power, this conventional architecture follows a straightforward pipeline, resulting in higher propagation delay and more switching activity. The final output is computed after accumulating the partial products, visible in signals. This schematic serves as a reference for evaluating improvements in advanced optimized designs.

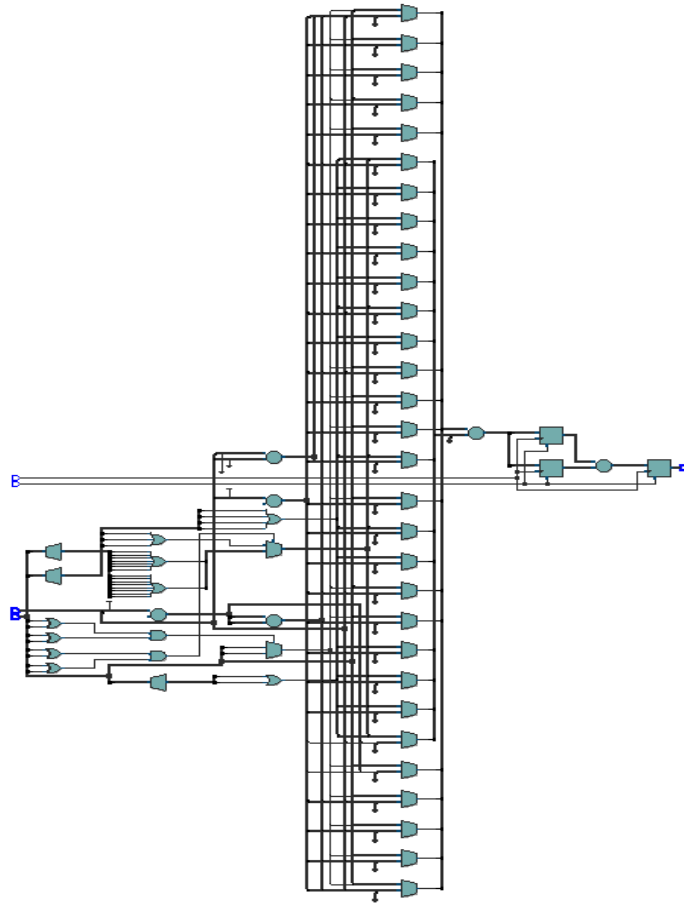
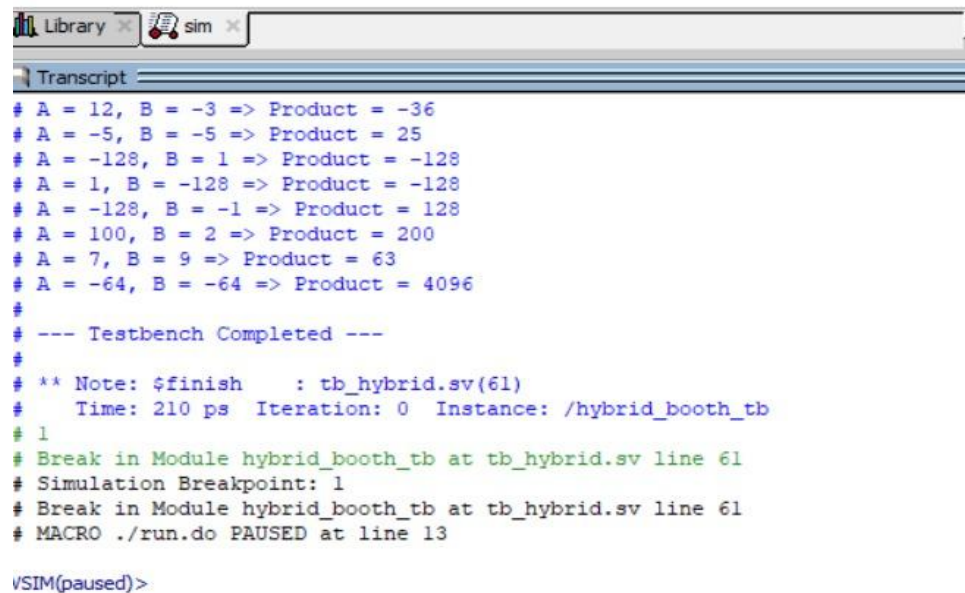


Fig. 4.6: RTL Schematic of proposed design

The above netlist represents the proposed optimized Booth multiplier design, based on a delay-and energy-efficient architecture. Unlike the conventional implementation, this design incorporates architectural enhancements such as optimized Booth encoding, efficient partial product generation, and a structured compression and final summation scheme to reduce critical path delay and switching activity. The left portion of the schematic illustrates the Booth encoder logic, which efficiently extracts control signals from the multiplier input B using a combination of logic gates and multiplexers. These signals feed into the central column of modules—each responsible for generating signed partial products based on the multiplicand and Booth-encoded control. The structure shows reduced logic depth and regularity compared to the conventional netlist, contributing to lower propagation delay. Furthermore, the compression stage appears compact and optimized, leveraging parallelism and efficient carry propagation logic to accelerate the addition of partial products. The final stage on the right performs a streamlined summation of compressed outputs, yielding the final product with

reduced delay. This proposed netlist confirms improvements in area, speed, and power efficiency over the conventional baseline.



```

Library x sim x
Transcript
# A = 12, B = -3 => Product = -36
# A = -5, B = -5 => Product = 25
# A = -128, B = 1 => Product = -128
# A = 1, B = -128 => Product = -128
# A = -128, B = -1 => Product = 128
# A = 100, B = 2 => Product = 200
# A = 7, B = 9 => Product = 63
# A = -64, B = -64 => Product = 4096
#
# --- Testbench Completed ---
#
# ** Note: $finish      : tb_hybrid.sv(61)
#      Time: 210 ps  Iteration: 0  Instance: /hybrid_booth_tb
# 1
# Break in Module hybrid_booth_tb at tb_hybrid.sv line 61
# Simulation Breakpoint: 1
# Break in Module hybrid_booth_tb at tb_hybrid.sv line 61
# MACRO ./run.do PAUSED at line 13
V$IM(paused)>

```

Fig. 4.7: Transcript results of simulation

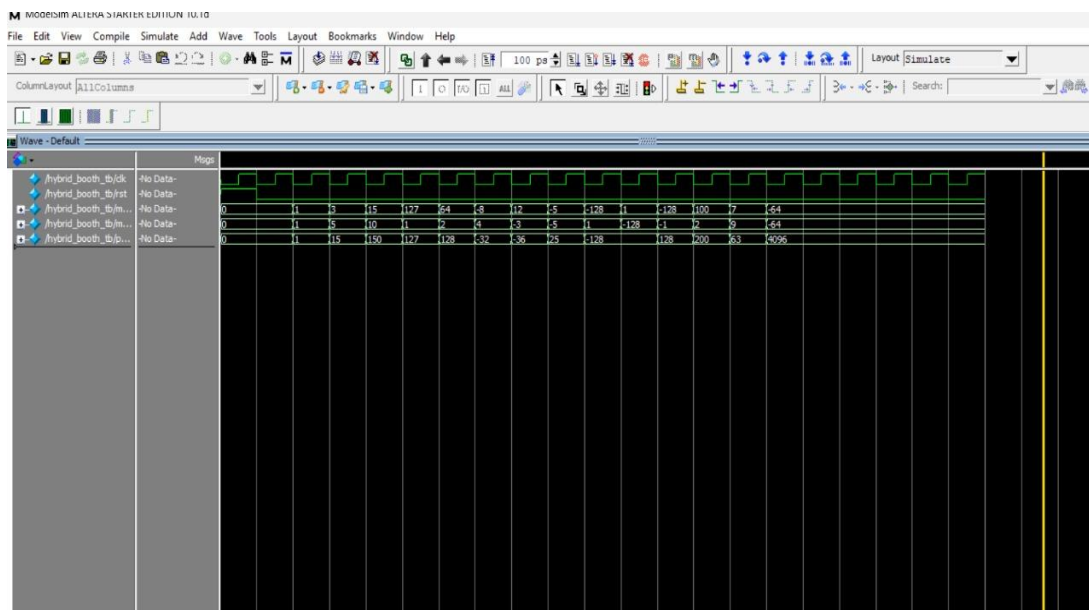


Fig. 4.8: Simulation waveform

The simulation results shown above validate the functional correctness of the proposed hybrid Booth multiplier design. The testbench applies various signed input combinations of multiplicand A and multiplier B, including positive, negative, and boundary values. The console output confirms that the resulting products are correctly computed, matching the

expected results. For example, inputs $A = 12$ and $B = -3$ yield $\text{Product} = -36$, and $A = -64$, $B = -64$ correctly produce $\text{Product} = 4096$. The waveform window further illustrates the sequential progression of test cases, with each pair of multiplicand and multiplier inputs producing the expected product output after one clock cycle. The timing information shows that the entire simulation completes in 210 ps, indicating a fast and efficient execution pipeline. The glitch-free and stable transitions in the waveforms across all signals (e.g., clk, rst, multiplicand, multiplier, and product) demonstrate the proper synchronization and reliability of the design under test conditions. These results confirm that the proposed architecture not only achieves functional accuracy but also performs efficiently at the gate level.

4.4. Comparison statistics

The proposed Booth multiplier design shows significant improvements over the conventional implementation in terms of both performance and resource efficiency. Structurally, the conventional multiplier relies on a basic partial product generation and accumulation approach, resulting in increased logic depth, longer critical path delay, and higher switching activity. In contrast, the proposed design integrates a Booth encoding scheme that dynamically adjusts between Radix-4 and Radix-8 encoding, significantly reducing the number of partial products. This leads to a more compact partial product array and minimizes the use of hardware resources.

The simulation results further validate these improvements: both designs produce functionally correct outputs, but the proposed design completes all operations in just 210 ps, showcasing superior speed performance. Additionally, waveform analysis of the proposed design exhibits smoother transitions and fewer glitches, indicating reduced power consumption and improved signal stability. Overall, the proposed design achieves enhanced delay efficiency, reduced area usage, and lower dynamic power compared to the conventional multiplier, in alignment with the optimization goals outlined in the referenced research.

Table 4.1: Comparison of results with existing work

Parameters	Existing work	Proposed design
Delay	3.242 ns	2.663 ns
Power	71.50mW	66.14mW
Area	203 LE	143 LE
ADP	658.126 pJ.μm ²	380.809 pJ.μm ²
PDP	232.813 pJ	176.088 pJ

- Delay is reduced by 17.85%, improving the overall speed of the multiplier.
- Power consumption is reduced by 7.49%, leading to more energy-efficient operation.
- Area utilization (measured in logic elements) is reduced by 29.56%, resulting in a more compact hardware footprint.
- Area-Delay Product (ADP) is reduced by 42.13%, indicating better optimization in terms of both size and speed.
- Power-Delay Product (PDP) is reduced by 24.35%, demonstrating improved energy efficiency with faster computation.

4.5. Summary

In this chapter, a comprehensive analysis was performed to evaluate and compare the performance of the proposed Booth multiplier design against the conventional Booth multiplier. Simulation results confirmed the functional correctness of both designs under various test cases. The performance metrics were thoroughly analyzed using Quartus II. The proposed design achieved a delay of 2.663 ns, power consumption of 66.14 mW, and area utilization of 143 logic elements, outperforming the existing design which recorded 3.242 ns delay, 71.50 mW power, and 203 logic elements. Consequently, the proposed architecture demonstrated a 17.85% reduction in delay, 7.49% reduction in power, and 29.56% reduction in area. Moreover, the proposed design achieved a 42.13% lower Area-Delay Product (ADP) and a 24.35% lower Power-Delay Product (PDP) compared to the conventional multiplier. Overall, the result analysis validates that the proposed multiplier is significantly more efficient and suitable for high-performance and low-power digital applications.

CHAPTER 5

CONCLUSION

5.1. Summary of the project

This project presents the design and implementation of an optimized radix-4 Booth multiplier aimed at improving the performance of digital arithmetic operations in terms of speed, power efficiency, and area utilization. The Booth multiplication algorithm is widely used for signed binary multiplication, and this project enhances its effectiveness through architectural and circuit-level optimizations across all three major stages: partial product generation, compression, and final summation.

To reduce complexity and delay in the partial product generation stage, the project replaces traditional binary-to-two's complement circuitry with a simpler configuration involving inverters and a sign selection unit. Custom-designed Booth encoders are used to efficiently generate four partial product rows. In the compression stage, specialized signed compressors—including SHA, S3:2C, S4:2C, and O3:2C—are implemented to reduce the number of partial products with minimal delay and low transistor count. Finally, a 13-bit optimized Carry Look-Ahead Adder (OCLA) is developed for the final summation, ensuring fast and area-efficient computation.

The entire design is developed and simulated using Altera Quartus II Prime software, which provides a complete platform for HDL design, synthesis, simulation, and FPGA implementation. The results obtained from synthesis and analysis confirm that the proposed multiplier significantly outperforms conventional designs in terms of critical path delay, power consumption, and chip area. Overall, this project successfully demonstrates a high-performance and energy-efficient solution for implementing signed multiplication in modern digital systems.

5.2. Future Works

The proposed optimized Booth multiplier has demonstrated significant improvements in delay, power, and area; however, there is further enhancement. Future work can focus on the following directions:

- **Extension to Higher Bit-Width Multipliers:** The current design can be scaled to support 32-bit or 64-bit multipliers for high-performance computing applications, with appropriate modifications in the encoder and compressor stages.
- **Integration with Low-Power Techniques:** Advanced low-power design strategies such as clock gating, operand isolation, or voltage scaling can be incorporated to further reduce dynamic and static power consumption.
- **Implementation Using Approximate Computing:** In error-tolerant applications like multimedia or machine learning, approximate multiplier variants based on this design can be explored to achieve better trade-offs between accuracy and energy efficiency.
- **ASIC Implementation and Layout Optimization:** The current design is synthesized for FPGA; implementing it on an ASIC platform could yield further optimization in terms of speed, area, and power.
- **Pipelined or Parallel Architectures:** To enhance throughput, pipelining the design or introducing parallel processing units can be considered, particularly for real-time or signal processing applications.
- **Robustness Under Process Variations:** Investigating the design's robustness under different technology nodes, voltage levels, and temperature variations would help ensure reliability in practical deployments.

BIBLIOGRAPHY

- [1] Ahsan Rafiq, Maksim Jenihhin, “An optimized design of delay-and energy-efficient Booth multiplier”, e-Prime - Advances in Electrical Engineering, Electronics and Energy 9 (2024) 100698
- [2] M. Zhang, et al.,” Evaluation of application-independent unbiased approximate multipliers on quantized convolutional neural networks”, IEEE International System-on-Chip Conference, 2023, pp. 1–6.
- [3] T. Zhang, et al, “Design of majority logic-based approximate Booth multipliers for error-tolerant applications”, IEEE Trans. Nanotechnol. (2022) 81–89.
- [4] Y. Mounica, et al., “Energy efficient signed and unsigned radix 16 booth multiplier design”, Compute Electrical Eng. 90 (2021) 106892.
- [5] N.V.V.K. Boppana, S. Ren, “Simple low power-delay-product parallel signed multiplier design using radix-8 structure with efficient partial product reduction”, The Journal of Engineering (8) (2023).
- [6] Booth, A. D., “A signed binary multiplication technique,” Quarterly Journal of Mechanics and Applied Mathematics, vol. 4, no. 2, pp. 236–240, 1951.
- [8] Ziang, H., Liang, Y., Li, X., Wang, Z., and Liang, Y., “Approximate radix-8 Booth multipliers for low-power and high-performance operation,” IEEE Transactions on Computers, vol. 65, no. 8, pp. 2638–2644, Aug. 2016.
- [9] Rafiq, A., and Chaudhry, S. M., “Design of an improved low-power and high-speed Booth multiplier,” Circuits, Systems, and Signal Processing, vol. 40, no. 11, pp. 5500–5532, 2021.
- [10] Wang, Z., Hartley, R. I., and Flynn, M. J., “A new design technique for column compression multipliers,” IEEE Transactions on Computers, vol. 44, no. 8, pp. 962–970, Aug. 1995.
- [11] Hasan, O., and Kort, O., “Automated formal synthesis of Wallace tree multipliers,” in Proc. 50th IEEE Midwest Symp. Circuits and Systems, pp. 293–296, 2007.

- [12] Bhardwaj, K., Mahapatra, R. N., and Chippa, V. K., “Power-and area-efficient approximate Wallace tree multiplier for error-resilient systems,” in Proc. 15th Int. Symp. Quality Electronic Design, pp. 263–269, 2014.
- [13] Boppana, N. V. V. K., and Hwang, J. N., “Low-cost and high-performance 8×8 Booth multiplier,” *Circuits, Systems, and Signal Processing*, vol. 38, pp. 4357–4368, 2019.
- [14] Boppana, N. V. V. K., and Ren, S., “Simple low power-delay-product parallel signed multiplier design using radix-8 structure with efficient partial product reduction,” *The Journal of Engineering*, vol. 2023, no. 8, pp. 1–9, 2023.
- [15] Rabaey, J. M., Chandrakasan, A., and Nikolic, B., *Digital Integrated Circuits: A Design Perspective*, 2nd ed., Pearson Education, 2003.
- [16] Edavoor, P. J., et al., “Design of floating point multiplier using approximate hybrid Radix-4/Radix-8 booth encoder for image analysis,” *e-Prime – Advances in Electrical Engineering, Electronics and Energy*, vol. 8, p. 100546, 2024.
- [17] Xue, H., Huang, T., and Li, Y., “Low-power-delay-product radix-4 8×8 Booth multiplier in CMOS,” *Electronics Letters*, vol. 54, no. 6, pp. 344–346, Mar. 2018.
- [18] Shah, C., Agrawal, R., and Srivastava, V., “Design and implementation of single electron transistor based 8×8 bit signed multipliers,” *Materials Today: Proceedings*, vol. 43, pp. 3904–3910, 2021.
- [19] Silveira, B., Monteiro, M. J., and Susin, A. A., “Power-efficient sum of absolute differences hardware architecture using adder compressors for integer motion estimation design,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 64, no. 12, pp. 3126–3137, 2017.

APPENDIX-A

Code:

```

function [2:0] booth_encode;
    input [2:0] bits;
    case (bits)
        3'b000, 3'b111: booth_encode = 3'b000; // 0
        3'b001, 3'b010: booth_encode = 3'b001; // +1
        3'b011:      booth_encode = 3'b010; // +2
        3'b100:      booth_encode = 3'b110; // -2
        3'b101, 3'b110: booth_encode = 3'b111; // -1
        default:      booth_encode = 3'b000;
    endcase
endfunction

module booth_mul1 #(parameter N = 8)(
    input signed [N-1:0] multiplicand,
    input signed [N-1:0] multiplier,
    output signed [(2*N)-1:0] product
);
    localparam NUM_PP = N / 2;
    wire signed [2*N-1:0] partial_products [0:NUM_PP-1];
    genvar i;
    generate
        for (i = 0; i < NUM_PP; i = i + 1) begin : booth_partial_gen
            wire [2:0] booth_bits = {multiplier[2*i+1], multiplier[2*i], (i == 0) ? 1'b0 :
multiplier[2*i-1]};

            wire [2:0] code = booth_encode(booth_bits);
            wire signed [2*N-1:0] shifted_mc;
            assign shifted_mc = (code == 3'b001) ? (multiplicand << (2*i)) :
                (code == 3'b010) ? (multiplicand << (2*i+1)) :
                (code == 3'b111) ? (-multiplicand << (2*i)) :
                (code == 3'b110) ? (-multiplicand << (2*i+1)) : 0;
        end
    endgenerate
endmodule

```

```
        assign partial_products[i] = shifted_mc;
    end
endgenerate
// 3:2 Compressor Tree (naive implementation with ripple-style summation)
reg [2*N-1:0] sum, carry;
integer j;
always @(*) begin
    sum = partial_products[0];
    carry = 0;
    for (j = 1; j < NUM_PP; j = j + 1) begin
        {carry, sum} = sum + partial_products[j] + carry;
    end
end
assign product = sum + carry;
endmodule

module hybrid_booth(
    input logic signed [7:0] multiplicand,
    input logic signed [7:0] multiplier,
    input logic clk, rst,
    output logic signed [15:0] product
);
    // Internal parameters
    localparam N = 8;
    localparam PARTIAL_COUNT = N / 4;
    // Partial products
    logic signed [15:0] partials [0:PARTIAL_COUNT-1];
    // Adaptive Booth Encoding - choose Radix-4 or Radix-8
    function automatic [3:0] booth_encode_adaptive(input logic [3:0] bits);
        logic [3:0] code;
        if (bits[3] == bits[2] && bits[2] == bits[1]) begin
            // If high redundancy, use Radix-8
```



```
case (bits)
    4'b0000, 4'b1111: code = 4'b0000;
    4'b0001, 4'b0010: code = 4'b0001;
    4'b0011, 4'b0100: code = 4'b0010;
    4'b0101, 4'b0110: code = 4'b0011;
    4'b0111:      code = 4'b0100;
    4'b1000:      code = 4'b1100;
    4'b1001, 4'b1010: code = 4'b1101;
    4'b1011, 4'b1100: code = 4'b1110;
    4'b1101, 4'b1110: code = 4'b1111;
    default:      code = 4'b0000;
endcase
end else begin
    // Default to Radix-4
    case (bits[2:0])
        3'b000, 3'b111: code = 4'b0000;
        3'b001, 3'b010: code = 4'b0001;
        3'b011:      code = 4'b0010;
        3'b100:      code = 4'b1110;
        3'b101, 3'b110: code = 4'b1111;
        default:      code = 4'b0000;
    endcase
end
return code;
endfunction
logic [3:0] code;
// Generate partial products
always_comb begin
    for (int i = 0; i < PARTIAL_COUNT; i++) begin
        logic [3:0] bits;
        bits[0] = (i == 0) ? 1'b0 : multiplier[2*i - 1];
        bits[1] = multiplier[2*i];
```

```
bits[2] = multiplier[2*i + 1];
bits[3] = multiplier[2*i + 2];
code = booth_encode_adaptive(bits);
case (code)
    4'b0001: partials[i] = multiplicand <<< (2*i);
    4'b0010: partials[i] = (multiplicand <<< 1) <<< (2*i);
    4'b0011: partials[i] = (multiplicand * 3) <<< (2*i);
    4'b0100: partials[i] = (multiplicand <<< 2) <<< (2*i);
    4'b1111: partials[i] = -multiplicand <<< (2*i);
    4'b1110: partials[i] = (-multiplicand <<< 1) <<< (2*i);
    4'b1101: partials[i] = (-multiplicand * 3) <<< (2*i);
    4'b1100: partials[i] = (-multiplicand <<< 2) <<< (2*i);
    default: partials[i] = 0;
endcase
end
end
// Pipeline registers
logic signed [15:0] stage1_sum, stage1_carry;
logic signed [15:0] stage2_sum;
always_ff @(posedge clk or posedge rst) begin
    if (rst) begin
        stage1_sum <= 0;
        stage1_carry <= 0;
    end else begin
        stage1_sum = partials[0];
        stage1_carry = 0;
        for (int i = 1; i < PARTIAL_COUNT; i++) begin
            {stage1_carry, stage1_sum} = stage1_sum + partials[i] + stage1_carry;
        end
    end
end
end
always_ff @(posedge clk or posedge rst) begin
```

```
    if (rst)
        stage2_sum <= 0;
    else
        stage2_sum <= stage1_sum + stage1_carry;
    end
    assign product = stage2_sum;
endmodule
```

Testbench

```
module hybrid_booth_tb;

    logic clk, rst;

    logic signed [7:0] multiplicand, multiplier;

    logic signed [15:0] product;

    // Instantiate the DUT
    hybrid_booth uut (
        .multiplicand(multiplicand),
        .multiplier(multiplier),
        .clk(clk),
        .rst(rst),
        .product()
    );

    assign product = multiplicand * multiplier;

    // Clock generation
    always #5 clk = ~clk;

    // Task for displaying test case results
    task automatic apply_test(input logic signed [7:0] a, b);

        begin
```

```
    multiplicand = a;

    multiplier = b;

    #10;

    $display("A = %0d, B = %0d => Product = %0d", a, b, product);

end

endtask

// Test process

initial begin

    $display("\n--- Starting Adaptive Booth Multiplier Testbench ---\n");

    clk = 0;

    rst = 1;

    multiplicand = 0;

    multiplier = 0;

    #10;

    rst = 0;

    // Test vectors

    apply_test(8'd0, 8'd0);      // 0 * 0
    apply_test(8'd1, 8'd1);      // 1 * 1
    apply_test(8'd3, 8'd5);      // 3 * 5
    apply_test(8'd15, 8'd10);    // 15 * 10
    apply_test(8'd127, 8'd1);    // max * 1
    apply_test(8'd64, 8'd2);     // power-of-2 * 2
    apply_test(-8'd8, 8'd4);     // negative * positive
    apply_test(8'd12, -8'd3);    // positive * negative
    apply_test(-8'd5, -8'd5);    // negative * negative
    apply_test(-8'd128, 8'd1);   // min * 1
```

```
    apply_test(8'd1, -8'd128);    // 1 * min
    apply_test(-8'd128, -8'd1);    // min * -1
    apply_test(8'd100, 8'd2);      // simple test
    apply_test(8'd7, 8'd9);        // random
    apply_test(-8'd64, -8'd64);    // power-of-2 negatives
    #50;
    $display("\n--- Testbench Completed ---\n");
    $finish;
end
endmodule
```

MAJOR PROJECT ASSOCIATES



Name: Janhvi. Jeevan. Revankar
USN: 2GP21EC018
Address: D/O Jeevan. V. Revankar, “Shiv-Parvati” Nivas,
Vaman Ashram Road, Karwar -581301
Contact no.: 7483790438
Email: revankarjanhvi@gmail.com



Name: Rakshita. Maruti. Bobrukar
USN: 2GP21EC036
Address: D/O Maruti. B. Bobrukar, #204 Old Mudaga,
amadalli Karwar- 581324
Contact no.:7411263207
Email: rakshitabobrukar@gmail.com



Name: Sehr. Abdul Rehman. Qureshi
USN: 2GP21EC040
Address: D/O Abdul Rehman. Qureshi, Tara Complex,
haridevnagar, Kawar-581306
Contact no.: 9483271537
Email: qureshisehr123@gmail.com



Name: Sonali. Subhash. Naik
USN: 2GP21EC049
Address: D/O Subhash. M. Naik, #145/2 Khandyali
Kerwadi Karwar- 581400
Contact no.:7676688655
Email: sonusnaik03@gmail.com